Functions

1. Introduction

In a modularized program the code has been written so that each task within the program has its own function. This is sometimes called the 'divide and conquer' approach to code development.

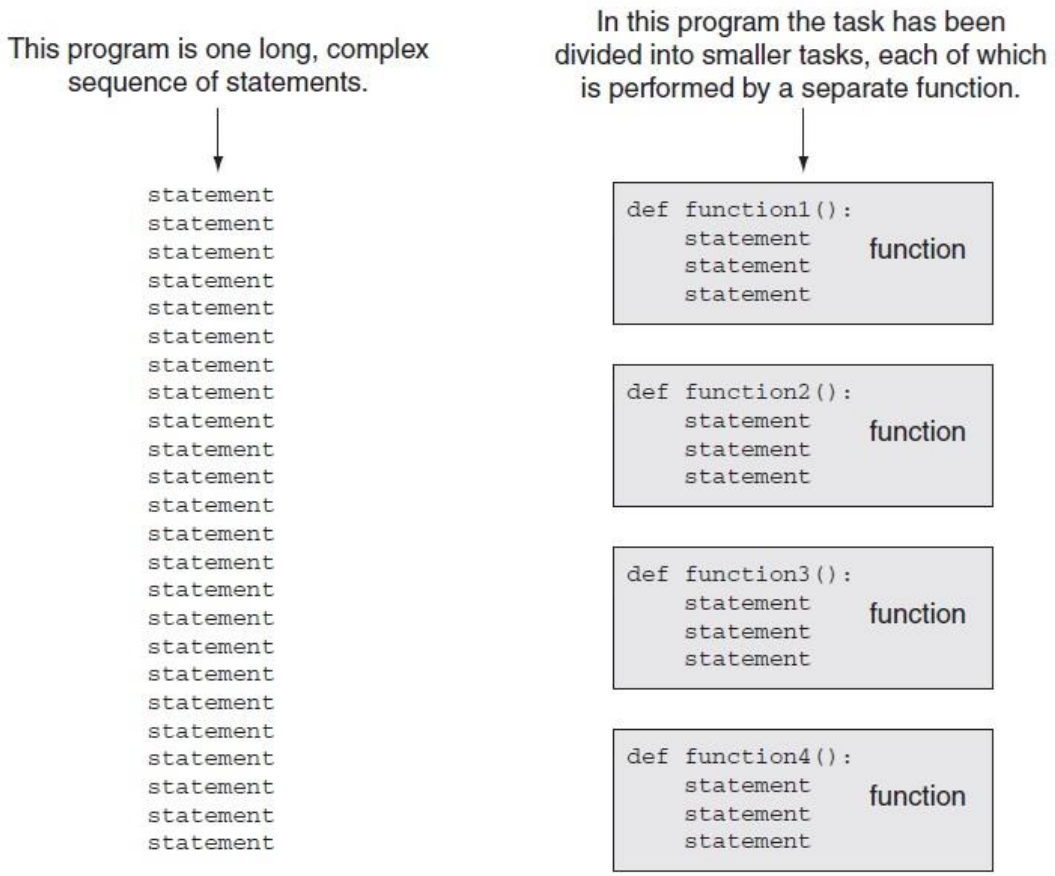

Figure 1 Using functions to 'divide and conquer' a task (taken from Gaddis, Starting Out with Python, chapter 5)

There are a number of benefits to using functions:

There are two types of function:

2. Defining and Naming Functions

There are some basic rules for naming a function.

- Key words cannot be used as a function name
- Cannot contain spaces
- The first character must be a letter or an underscore
- All other characters must be either a number, letter or underscore
- Upper and lower case letters are distinct

Try to choose meaningful function names (e.g. `calculate_pay` or `print_address`) as this helps the readability of the code.

In Python syntax a function definition is written as

```
def function_name():
    statement
    statement
    …
```

To execute a function it must be **called**.

One function can call another function, and a function can even call itself (via a process called recursion).

It is common for a program to have a main function that calls the other functions. This main function is then said to contain the **mainline logic** (or overall structure) of the program.

Example

function_demo.py

```
# This program demonstrates a function.
# First, we define a function named message.

def message():
    print('We are the Knights')
    print('who say Nee.')

# Call the message function.

message()
```

3. Flowcharts, Hierarchy Charts, and Top-Down Design

In a flowchart a function call is shown as a rectangle with vertical bars at each side, with the function name written in the box.

Separate flowcharts are then drawn for each function in the program.

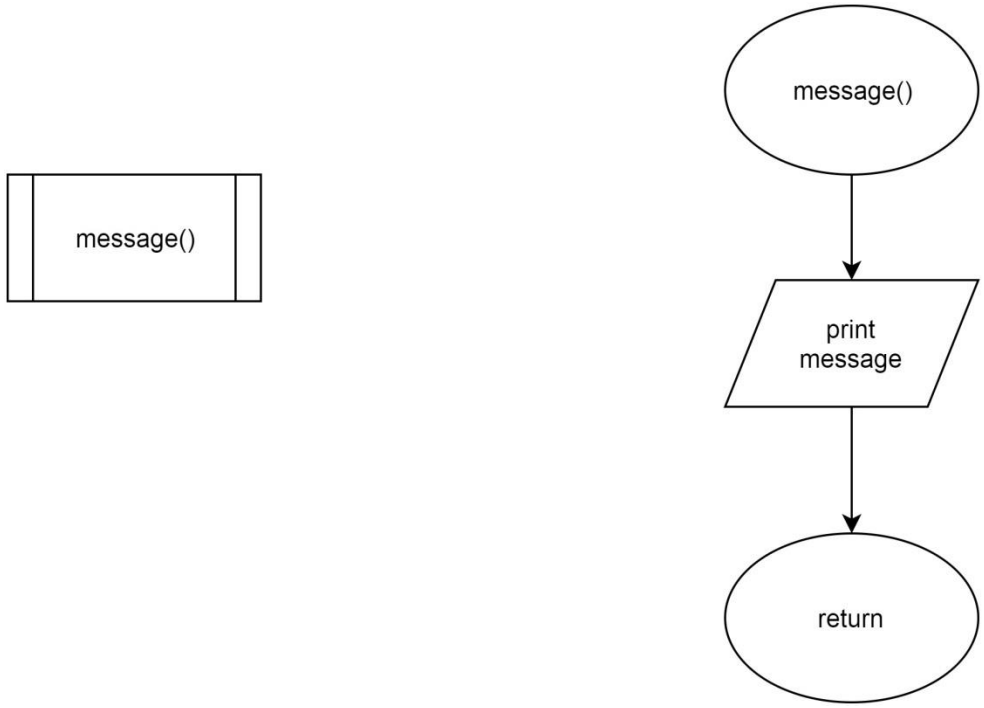

Figure 2 The flowchart symbol for a function, and a function flowchart

In top-down design a programming problem is dealt with as follows

1. The overall program task is broken into subtasks for which functions can be written.
2. Each subtask is checked to see if it can be further subdivided into other functions.
3. This process is repeated until no further sensible subtasks can be identified
4. The code is then written.

A hierarchy chart (also called a structure chart) can be drawn to show which functions call which.

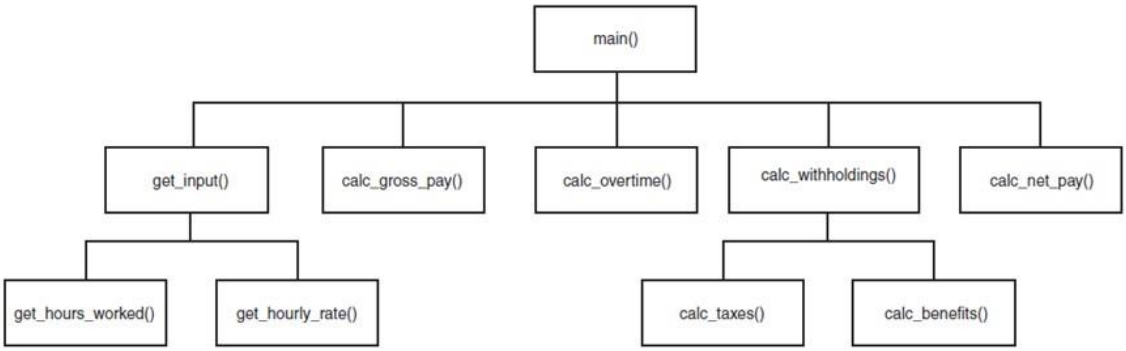

Figure 3. Example of a hierarchy diagram (taken from Starting Out with Python by Gaddis)

A useful way to think of a function is as an little independent program, with one function not 'seeing' the others internal workings. So two functions could have variables with the same names, but each function does not 'see' the other function's variables so there is no confusion. The variables defined inside a function are called local variables because they local only to that function.

4. Function Arguments

Examples

pass_arg.py

```
# This program demonstrates an argument being
# passed to a function.

def main():
    value = 5
    show_double(value)

# The show_double function takes the argument number
# and displays double its value.
def show_double(number):
    result = number * 2
    print(result)
# end of function show_double

# Call the main function.
main()
```

multiple_args.py

```
# This program demonstrates a function that accepts
# two arguments.

def main():
    print('21 divided by 23 is')
    show_division(21, 23)

# The show_division function accepts two arguments
# and displays their ratio .
def show_division(num1, num2):
    result = num1 / num2
    print(result)

# Call the main function.
main()
```

5. Keyword Arguments

In the previous example the arguments were passed by their order on the list. This is the standard and most sensible way to pass arguments.

However, Python is quite flexible in how it allows arguments to be passed, and they can also be passed as what is called a keyword argument, where the programmer explicitly specifies which values should be assigned to which arguments. This is illustrated with a variant on `multiple_args.py` below:

```
# This program demonstrates a function that accepts
# two arguments.

def main():
    print('21 divided by 23 is')
    show_division(num2 = 23, num1 = 21)

# The show_division function accepts two arguments
# and displays their ratio .
def show_division(num1, num2):
    result = num1 / num2
    print(result)

# Call the main function.
main()
```

6. Global Constants[1]

If a constant or variable is created inside a function then it is local to that function and can be accessed only by statements inside the function.

Example

```
# The following is used as a global constant to represent
# the contribution rate.
CONTRIBUTION_RATE = 0.05

def main():
    gross_pay = float(input('Enter the gross pay: '))
    bonus = float(input('Enter the amount of bonuses: '))
    show_pay_contrib(gross_pay)
    show_bonus_contrib(bonus)
# end of function main

# The show_pay_contrib function accepts the gross
# pay as an argument and displays the retirement
# contribution for that amount of pay.
def show_pay_contrib(gross):
    contrib = gross * CONTRIBUTION_RATE
    print('Contribution for gross pay: $', \
          format(contrib, ',.2f'), \
          sep='')
# end of function show_pay_contrib

# The show_bonus_contrib function accepts the
# bonus amount as an argument and displays the
# retirement contribution for that amount of pay.
def show_bonus_contrib(bonus):
    contrib = bonus * CONTRIBUTION_RATE
    print('Contribution for bonuses: $', \
          format(contrib, ',.2f'), \
          sep='')
# end of function show_bonus_contrib

# Call the main function.
main()
```

[1] Note there are also things called global variables – which can be changed by any statement within the program file. However, I am not even going to tell you about these, as they make programs difficult to follow and difficult to debug, and are a cause of naughtiness and poor behaviour in children. (Admittedly I can't back that last bit up with evidence.)

7. Value Returning Functions & Libraries

At the beginning of this chapter we noted that there are two types of functions; **void functions** and **value returning functions**. The functions we have looked at so far have all been void functions i.e. they execute a set of statements and then stop. A value returning function does this too – but it also returns a value back to the part of the program that called it.

Python has a standard library of functions some of which are 'built in'. We have already used some of these – such as `print, input` and `range`. Other sets of functions are stored in **modules** which although they come with Python when you install it on your computer have to be imported at the top of your program. You have already done this with the `import turtle` command. A function in a module is called using what is called 'dot notation' which is of the form `module_name.function_name()`. Again, you have already seen this with commands in turtle such as

```
turtle.pencolor('blue')
```

8. The `random` Library

Random numbers are used in computer games, simulations and statistical modelling. Python provides a library of functions which produce random numbers[2] called `random`.

To import the module we put

```
import random
```

at the top of our code. You can also experiment with the functions in `random` in the interactive mode. The random functions we will look at are all **value returning functions** – i.e. they all return a value (a random number) back.

[2] Actually that is a lie. A computer can't produce a genuinely random number, as the 'random number' it generates is produced as the result of a sequence of mathematical formulae (which are deterministic). What the functions produce are **pseudo-random numbers**, which are numbers which do a very good job of *looking* like they are genuinely random.

```
random.randint(1,100)

random.randrange(5)

random.randrange(5,10)

random.randrange(0,121,10)

random.random()

random.uniform(1.0,3.0)
```

Example

coin_toss.py

```
# This program simulates 25 tosses of a coin.
import random

def main():
    tosses = 25
    coin(tosses)
#end of main function

def coin(tosses):
    for toss in range(tosses):
        # Simulate the coin toss.
        if random.randint(1, 2) == 1:
            print('Heads')
        else:
            print('Tails')
        # end of if statement
     #end of for loop
#end of coin function


# Call the main function.
main()
```

9. Writing Value Returning Functions

In terms of programming the only difference between a void function and a value returning function is that a value returning function includes a `return` statement with the Python syntax looking like:

```
def function_name():

    statement

    statement

    …

    return expression
```

Example

temp_converter.py

```
# this program does a temperature conversion

def main():
    #get temperature in centigrade
    temp=float(input('Enter the temperature in centigrade: '))

    #get temp. in Farenheit
    temp_far=fahrenheit(temp)

    #display result
    print('The temperature in Fahrenheit is: ',temp_far)
# end of main function

def fahrenheit(temp):
    #convert centigrade to fahrenheit
    #using standard formula
    result = temp*1.8 +32
    return result
# end of fahrenheit function

# call main function
main()
```

10. Storing Functions in Modules

A module is simply a file that contains the Python code for a number of functions. Thus, if you have developed a number of related functions that you are likely to use in different programs, a sensible thing to do is to place them all in one file, and then use the `import` command.

Thus, say we have written two functions `area(radius)` and `circumference(radius)` which return the area and circumference of a circle given its radius. We place both of these is the file `circle.py`.

To use these in some other program we simply put the command

```
import circle
```

at the top of our code. The functions are then called as

`circle.area(radius)` and `circle.circumference(radius)`

The only thing you need to remember is that `circle.py` is in the same folder as the program that calls it.

Supportive Reading

Tony Gaddis, *Starting Out with Python*, 5/e Pearson, 2022, Chapter 5.